УНИВЕРСИТЕТ ИТМО

Дисциплина Встроенные системы

Отчет по мастер классу 3

Программирование микроконтроллеров
с операционной системой FreeRTOS

Игра Змейка на стенде SDK1.1M

Вариант 1

Выполнили: Тимошкин Роман Вячеславович

Муравенко Григорий Павлович

Группа: P3330

Преподаватель: Быковский Сергей Вячеславович

Санкт-Петербург
2026

1 Цель и задачи работы

Цель работы — освоить программирование микроконтроллера с операционной системой FreeRTOS и реализовать игру «Змейка» на учебном стенде SDK1.1M с матричной клавиатурой и OLED дисплеем. За основу взят проект SDK_FreeRTOS с использованием STM32 HAL и CMSIS RTOS v1.

По варианту 1 змейка перемещается по полю, растет при поедании еды и набирает очки. Клавиатура задает направление, паузу и перезапуск. Состояние игры выводится на OLED; столкновение со стеной или телом завершает игру.

Задачи работы

- Настроить обмен с PCA9538 и OLED по общей шине I2C1.
- Организовать опрос клавиатуры 4 × 3 с подавлением дребезга и выделением новых нажатий.
- Разделить ввод, игру, отрисовку и звук на четыре задачи FreeRTOS.
- Использовать три очереди и один общий мьютекс для шины I2C1.
- Реализовать рост, счет, ускорение, паузу, перезапуск и обработку столкновений.

Используемые устройства и подключения

Устройство или сигнал	Подключение и назначение
Микроконтроллер STM32F407	Cortex M4; HSI 16 МГц
I2C1 SCL и SDA	PB8 и PB9; скорость 100 кГц
PCA9538	Адрес 0x71; клавиатура 4 × 3
OLED 128 × 64	Адрес 0x3C; вывод поля и счета
SysTick TIM1 TIM2	Такты ОС; HAL; зуммер PA5
Программатор SDK1.1M	FTDI/JTAG; загрузка прошивки

В HAL адреса заданы со сдвигом: 0xE2 для PCA9538 и 0x78 для OLED. Настройка строк повторяет SDK_Keyboard. SysTick обеспечивает такты FreeRTOS, а TIM1 — миллисекундный счетчик HAL. TIM2_CH1 на PA5 управляет зуммером по примеру SDK_Buzzer; USART1 сохранен.

2 Диаграмма последовательностей

Голубым показаны программные сущности, оранжевым — аппаратные. Input, Game, Display и Sound — задачи FreeRTOS; i2c_mutex — мьютекс. commands, frames и sounds — очереди, подписанные на стрелках обмена. main — функция, FreeRTOS — ОС.

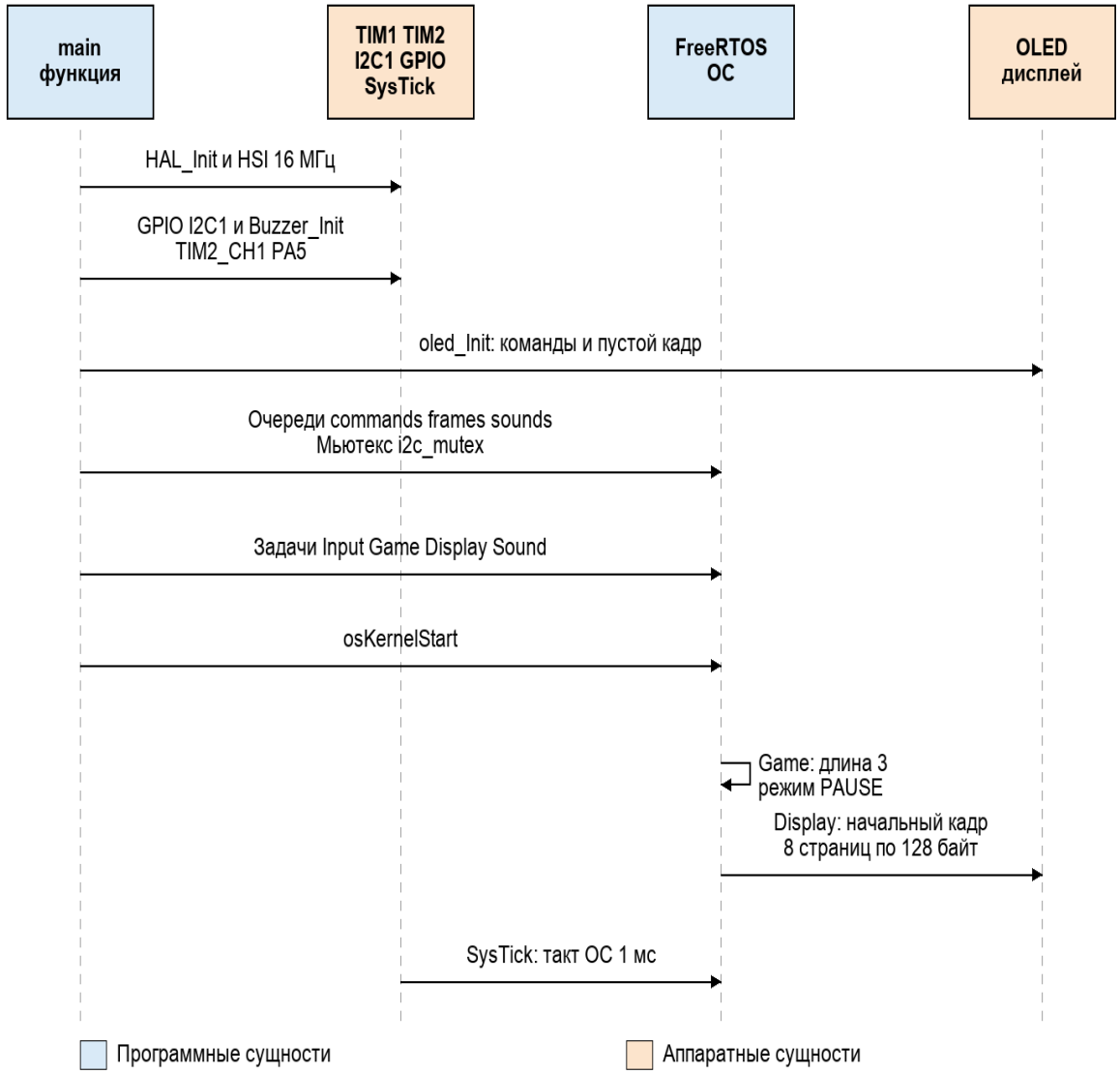


Рисунок 1 — Инициализация периферии и запуск задач FreeRTOS

До старта ОС main настраивает периферию и OLED. После osKernelStart задача Game создает змейку и включает паузу, а Display показывает начальное состояние. TIM1 и SysTick используют раздельные временные базы.

2 Диаграмма последовательностей продолжение

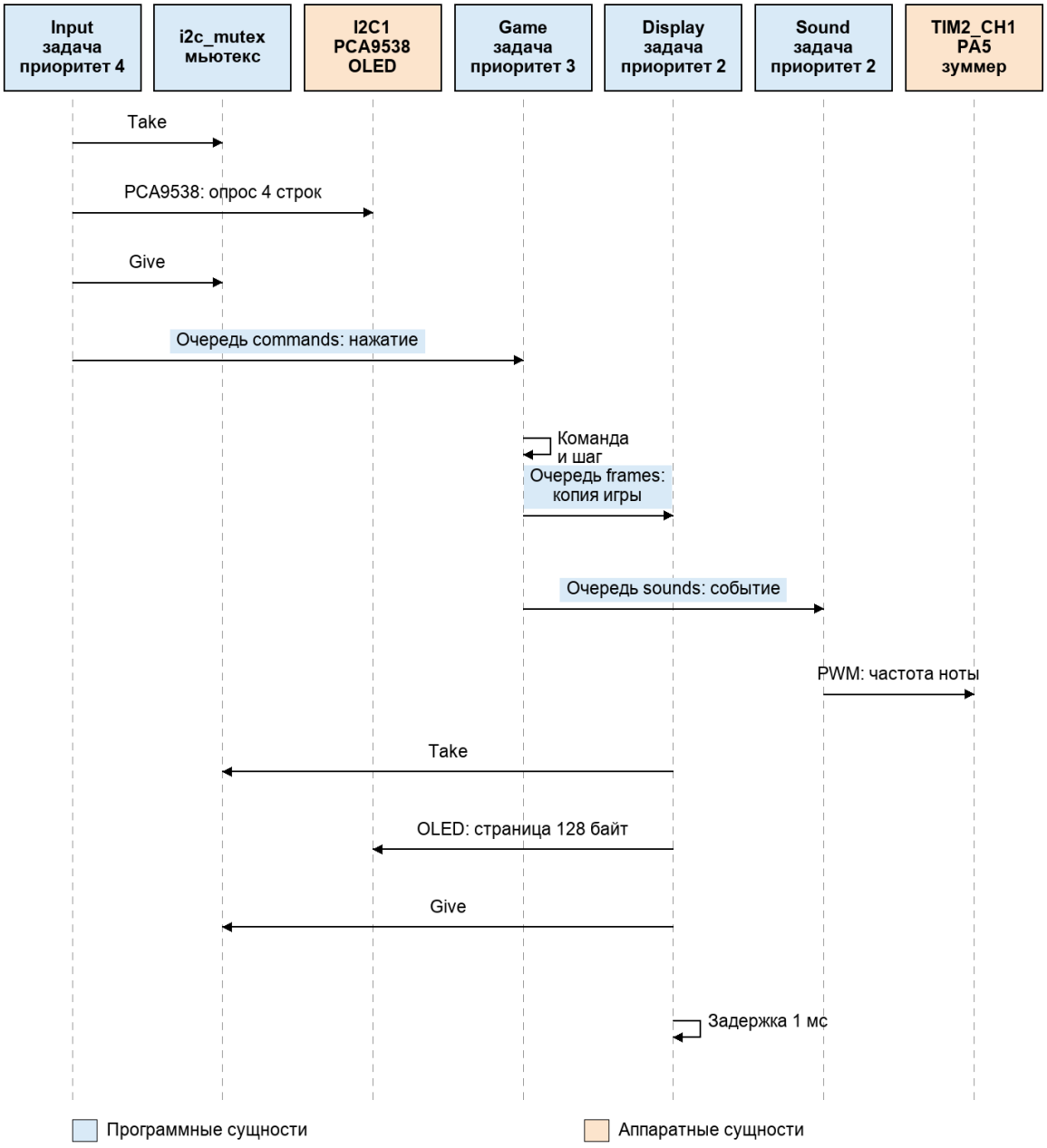


Рисунок 2 — Опрос клавиатуры передача команд и обновление OLED

Очередь commands передает команды из Input в Game; frames — копию игры из Game в Display; sounds — события из Game в Sound. Input и Display используют i2c_mutex для обмена по I2C1. Задача Sound управляет аппаратным TIM2_CH1 на PA5.

3 Описание процессов на диаграмме

Инициализация и отсчет времени

После сброса HAL_Init настраивает HAL и временную базу TIM1. SystemClock_Config выбирает HSI 16 МГц. Затем настраиваются GPIO, USART1 и I2C1 на PB8 и PB9 со скоростью 100 кГц. Драйвер OLED передает команды инициализации и очищает экран.

MX_FREERTOS_Init создает четыре задачи, три очереди и один мьютекс. Input имеет приоритет AboveNormal, Game — Normal, Display и Sound — BelowNormal. SysTick задает такт ОС 1 мс. Input и Game используют vTaskDelayUntil с периодом 10 мс; задачи ожидания не занимают процессор.

Опрос клавиатуры и подавление дребезга

Задача Input вызывает keyboard_read и последовательно проверяет четыре строки. Перед каждой строкой записываются POLARITY_INVERSION, OUTPUT_PORT и CONFIG, как в SDK_Keyboard. Маски 0xFE, 0xFD, 0xFB и 0xF7 выбирают строку, переводимую в низкий уровень; остальные строки остаются входами.

Из INPUT_PORT выделяются биты 4–6. Строка и столбец определяют один из 12 битов маски клавиатуры. Три одинаковых чтения подтверждают изменение. В очередь commands отправляется событие нового нажатия; несколько одновременно нажатых клавиш игнорируются. Ошибка I2C не считается отпусканием.

commands содержит восемь элементов uint8_t; при заполнении новая команда отбрасывается без блокировки Input. Game — единственный владелец состояния игры. frames содержит одну копию SnakeGame: xQueueOverwrite оставляет последний кадр, поэтому отрисовка не задерживает движение. sounds хранит восемь событий uint8_t; при заполнении звук пропускается. Стеки Input, Game, Display и Sound — 256, 256, 512 и 256 слов.

Состояния игры и размещение еды

На старте длина змейки равна трем клеткам и включена пауза. Клавиша 5 запускает или приостанавливает движение; * начинает новую игру на паузе. Направления 2, 4, 6 и 8 означают вверх, влево, вправо и вниз. Разворот на 180° запрещен; за один шаг принимается только один поворот.

Поле имеет размер 30 × 12 клеток. Еда появляется только во внутренних свободных клетках, с отступом в одну клетку от краев. За еду начисляется очко и добавляется сегмент. Интервал движения уменьшается с 240 до 90 мс. Столкновение означает проигрыш; заполнение всех внутренних клеток — победу.

3 Передача данных между задачами и периферией

Обмен по I2C и вывод на дисплей

Клавиатура и OLED используют блокирующие HAL_I2C_Mem_Read и HAL_I2C_Mem_Write на общей I2C1. Мьютекс с наследованием приоритета исключает одновременный обмен. Input удерживает его на время сканирования, Display — на время одной страницы. DMA и прерывания клавиатуры не используются.

Display ожидает frames и формирует буфер 1024 байта. В верхней строке шрифтом 7 × 10 показаны состояние и счет; ниже — рамка, змейка и еда. Буфер передается восемью страницами по 128 байт. Между страницами мьютекс освобождается и выполняется задержка FreeRTOS 1 мс. Буфер принадлежит только Display. Sound — единственный владелец TIM2: старт, пауза, перезапуск, еда, проигрыш и победа имеют разные сигналы. Частота PWM задает высоту тона, заполнение — 50%; нулевая частота отключает звук. vTaskDelay задает длительность нот, освобождая процессор.

4 Руководство пользователя

Запуск программы

1. Подключить питание стенда и USB DBG. Импортировать SDK_Snake в STM32CubeIDE и собрать Debug либо выполнить ./lab.sh build.
2. Выполнить ./lab.sh flash: скрипт соберет, загрузит, проверит и запустит прошивку через FTDI/JTAG с SDK11M.cfg.
3. Проверить появление PAUSE 0, рамки, змейки и еды. Раскладка сверху вниз: 1 2 3 / 4 5 6 / 7 8 9 / * 0 #.
4. Проверить звуки старта, паузы, еды и проигрыша. Нажать 5 для старта. Управлять клавишами 2, 4, 6 и 8, отпуская каждую перед следующим нажатием. Повторное 5 включает паузу.

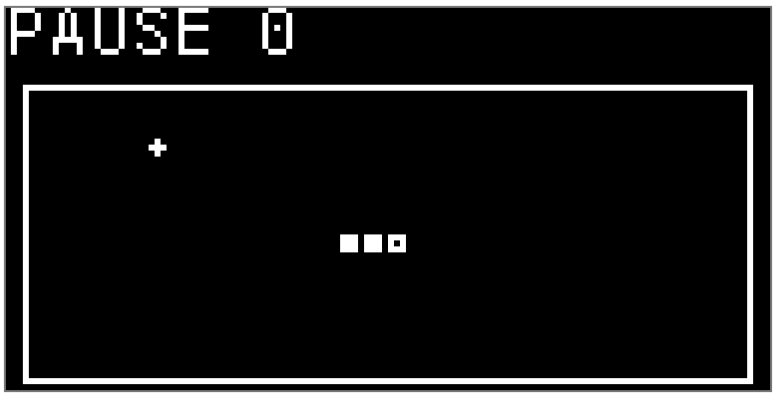
Клавиша	Действие
2 / 4 / 6 / 8	Вверх / влево / вправо / вниз
5	Старт или переключение паузы
*	Новая игра на паузе

Удерживать направление не требуется. После проигрыша направления и 5 не действуют: нажать * для новой игры, затем 5 для старта. В правую сторону при старте змейка движется автоматически; 4 не разворачивает ее назад. Еда отмечена крестиком, голова — квадратом с темным центром.

4 Демонстрация основных функций

Изображения получены программной моделью OLED: выполнены snake.c, текущая отрисовка, oled.c и fonts.c; передача I2C заменена захватом буфера. Для демонстрации роста задано положение еды перед головой. Модель не воспроизводит физические сигналы и планирование задач.

Начальное состояние

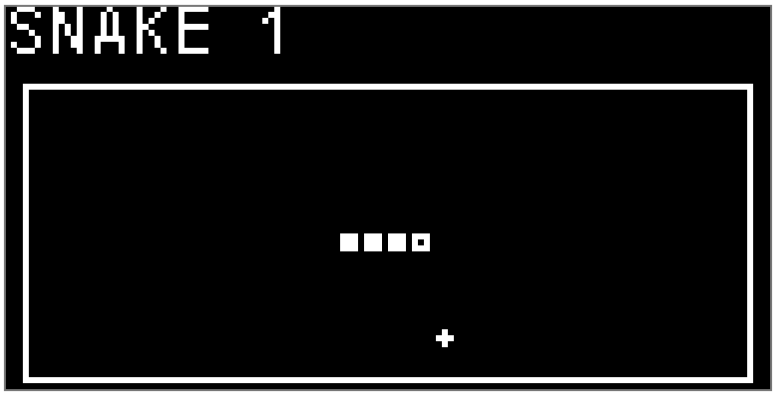


Программная модель OLED 128 x 64

Рисунок 3 — Начальная змейка и ожидание нажатия 5

После запуска показаны PAUSE 0 и змейка длиной три клетки. Движение начинается только после 5. Такая же начальная пауза устанавливается после перезапуска клавишей *.

Рост после поедания еды



Программная модель OLED 128 x 64

Рисунок 4 — Увеличение длины и счета после поедания еды

При переходе головы на еду счет становится равным 1, длина увеличивается до четырех клеток. Следующая еда выбирается внутри поля, вне тела змейки и вне крайних строк и столбцов.

4 Пауза и завершение игры

Пауза во время игры

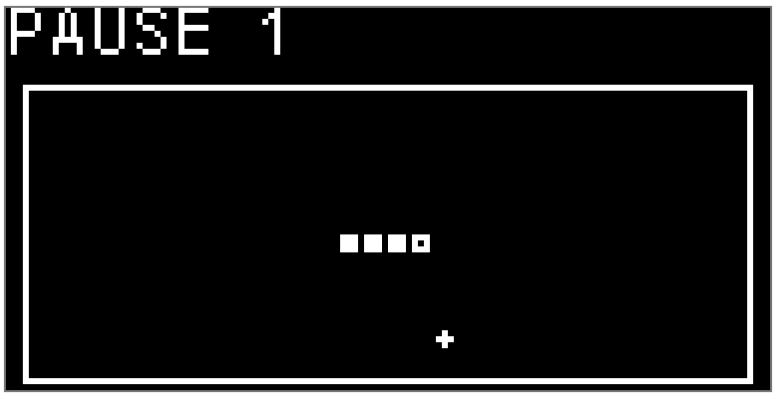


Рисунок 5 — Пауза с сохранением игрового поля и счета

Нажатие 5 переводит игру в PAUSE. Координаты сегментов, положение еды и счет сохраняются. Повторное нажатие продолжает движение; пауза не блокирует опрос клавиатуры.

Столкновение со стеной



Рисунок 6 — Проигрыш после столкновения со стеной

При выходе следующего шага за границу устанавливается LOST *. Змейка остается в последнем допустимом положении. Нажатие * сбрасывает длину и счет и создает новую игру на паузе; после 5 можно играть снова.

4 Контрольные примеры и исходный код

Контрольные примеры приведены в таблице. Каждая клавиша означает отдельное нажатие и отпускание. Сценарии роста и столкновений проверены автоматическими тестами игровой логики.

Клавиши или действие	Ожидаемый результат
Запуск или *	PAUSE 0; длина 3
5 при начальной паузе	Движение вправо
2 / 6 / 8 / 4	Изменение направления
4 при движении вправо	Разворот не выполняется
Два поворота до шага	Принимается только первый
Поедание еды	Счет +1; длина +1
5 во время движения	Поле и счет сохраняются
Повторное 5 на паузе	Движение продолжается
Стена или тело	LOST *; движение прекращается
Внутренние клетки заняты	WIN *; еда на краях не появляется

Сборка ARM GCC завершена: text 24496 байт, data 112 байт, bss 20256 байт. Тесты прошли с контролем памяти и неопределенного поведения. Проверено сопоставление всех 12 клавиш и обработка ошибок I2C. Управление подтверждено на стенде ранее; новый звук требует проверки после загрузки. Кадры получены программной моделью.

5 Ссылка на репозиторий

https://github.com/rdydygin/ITMO/tree/main/Semester5/EmbeddedSystems/MasterClass3

Вывод

Реализована «Змейка» на FreeRTOS. Ввод, игровая логика, вывод и звук выполняются четырьмя задачами. Очереди передают команды и копии состояния, мьютекс защищает I2C1. Игра поддерживает рост, счет, ускорение, паузу и перезапуск. Еда размещается внутри поля; столкновения обрабатываются без выхода за границы массива.